EPHEC
ÉCOLE SUPÉRIEURE
ENSEIGNEMENT POUR ADULTES

Année académique **2026-2027**

Support de cours - Programmation orientée objet - Java

# Module JUN

## Tests unitaires avec JUnit

Écrire le code qui vérifie le code

EPHEC — Bachelier en informatique, niveau 2

Depuis le début de l'année, un test vous dit si votre code est juste. Ce module vous apprend à écrire ce test vous-mêmes : du code qui dira, tout seul et à chaque lancement, que quelque chose s'est cassé.

Le module ne compte que trois exercices, et ce chiffre est trompeur. Quatre objectifs de l'examen ne reposent que sur lui : écrire la classe de test d'une classe écrite par quelqu'un d'autre, écrire une classe et ses tests, repérer un défaut grâce aux tests, et justifier les cas que l'on a choisi de tester. Chaque session consacre sa troisième question à une classe de test.

> **À la fin de ce module, vous saurez**
>
> - écrire une classe de test `JUnit`, au bon endroit, avec des tests en trois temps ;
> - choisir l'assertion la plus précise, et vérifier qu'une exception est levée ;
> - traduire une documentation en liste de tests, y compris pour les cas d'échec ;
> - choisir les cas à tester par familles et par frontières, et justifier chaque test par le défaut qu'il attrape ;
> - lire un test rouge, et décider qui a tort, du code ou du test ;
> - rendre testable une classe qui dépend d'une fenêtre, de la date du jour ou du hasard.

## JUN §1 — À vous d'écrire les tests

### Qui écrit quoi

Jusqu'ici, le test était fourni et vous écriviez le code. Dans ce module, les rôles s'inversent : c'est à vous d'écrire les tests.

| Exercice | Fourni | À écrire |
|---|---|---|
| jusqu'ici | le test | le code |
| `JUN.1` | le test | le code, une dernière fois |
| `JUN.2` | quatre tirelires | la vérification |
| `JUN.3` | la classe `Moyenne` | sa classe de test |
| l'examen | une classe d'un tiers | sa classe de test |

Lisez ce tableau de haut en bas : la colonne de droite change de nature. À partir de `JUN.2`, le code est déjà écrit, et souvent correct ; ce que vous rendez, c'est ce qui le vérifie. `JUN.1` est la charnière : le test y est encore fourni, mais il ne sert plus seulement à dire vert ou rouge. Il est l'énoncé, et vous apprenez à le lire.

Conséquence pratique : le vert n'est plus la preuve que vous avez fini. Dans `JUN.3`, la classe de test fournie contrôle la *forme* de la vôtre, pas sa valeur. Personne ne peut la contrôler à votre place, et c'est exactement ce que l'examen évalue.

### Un programme qui affiche ne vérifie rien

```
public static void main(String[] args)
        throws SalleCompleteException {
    Salle salle = new Salle(100);

    salle.reserver(2);

    System.out.println(salle.placesLibres());   // 98 ?
}
```

C'est ainsi que vous avez vérifié vos programmes jusqu'ici : on lance, on lit ce qui s'affiche, on décide si c'est juste. Cela marche une fois. Personne ne relance quarante programmes après chaque modification, en relisant chaque ligne et en se souvenant de ce qu'elle devait afficher.

Un test fait la même chose, à une différence près : il contient la réponse attendue. Il compare lui-même, et ne dit que vert ou rouge. C'est le « S » de F.I.R.S.T.[1], *auto-vérifiant*, dans `CC-TST.3`.

La salle de spectacle servira d'exemple pour tout le module. Elle a une capacité, on y réserve des places, et elle refuse ce qu'elle ne peut pas servir.

> **Démonstration** - `Demo_JUN` : rouge, vert, puis nettoyer
>
> La démonstration écrit un test avant la classe qu'il vérifie, le regarde échouer, puis le fait passer avec le code le plus bête possible. Un test de plus, et le code doit s'améliorer. Un cas inattendu apparaît, que seul le fait de chercher quoi tester a fait découvrir. Deux idées à retenir : un test qu'on n'a jamais vu échouer ne prouve rien ; et un test sans assertion passe au vert quoi qu'il arrive, ce que la démonstration montre exprès, parce que c'est le défaut le plus fréquent dans les copies d'examen.

## JUN §2 — Ce qu'un test contient

### Où vit une classe de test

```
package be.ephec.java.spectacle;

import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.assertEquals;

class SalleTest {

    // une methode par comportement verifie
}
```

- **Dans `src/test/java`, sous le même package que la classe testée.** Les deux arborescences se superposent : la classe de test voit sa classe comme une voisine (voir `PKG §1`).
- **Son nom est celui de la classe, suivi de `Test`.**
- **`JUnit` n'est visible que de ce côté.** Le projet Maven du cours ne laisse `JUnit` entrer que dans `src/test` ; tenter de l'importer dans `src/main` ne compile pas, et c'est voulu (`TEC-MVN.2`).

[1] F.I.R.S.T. réunit, en cinq mots anglais, les qualités d'un bon test : *Fast*, rapide, parce qu'une suite lente n'est plus lancée ; *Independent*, indépendant, parce qu'aucun test ne suppose qu'un autre a tourné avant lui ; *Repeatable*, répétable, parce qu'il donne le même résultat sur toutes les machines et dans six mois ; *Self-validating*, auto-vérifiant, parce qu'il dit vert ou rouge sans qu'on lise une sortie ; *Timely*, écrit tôt, idéalement avant le code. Le mémento Clean Code les détaille en `CC-TST.3`.

Notez bien : le code « utile » que vous produisez, celui qui partira en production, se trouve dans le dossier `src/main/java` ; le code de test, qui ne doit pas partir en production, est dans `src/test/java`.

L'`import static` est nouveau. Il importe une méthode statique, ici `assertEquals` de la classe `Assertions`, pour l'appeler sans écrire le nom de la classe devant. Toutes les assertions de `JUnit` 5 vivent dans `org.junit.jupiter.api.Assertions`.

Dans IntelliJ, `Ctrl+Shift+T` depuis la classe propose de créer sa classe de test au bon endroit. À l'examen, si les noms de `JUnit` restent en rouge, l'énoncé rappelle l'option qui l'ajoute au projet.

### `@Test`, puis une assertion

```
@Test
void uneSalleNeuveEstEntierementLibre() {
    Salle salle = new Salle(100);

    int libres = salle.placesLibres();

    assertEquals(100, libres,
            "Une salle neuve de 100 places doit etre entierement libre");
}
```

L'annotation `@Test` fait d'une méthode un test, que `JUnit` lance seul. Une méthode de test ne reçoit rien et ne retourne rien. Il n'y a pas de `main` : `JUnit` cherche toutes les méthodes annotées `@Test` et les lance une à une.

Les trois temps se lisent dans le code :

1. *préparer* la salle ;
2. *agir* en demandant les places libres ;
3. *vérifier* le résultat.

Une ligne vide les sépare, et c'est la forme à donner à vos propres tests, à l'examen comme ailleurs. Vous ne la retrouverez pas dans tous les tests fournis avec les exercices : beaucoup ne vérifient que la forme d'une classe, en une ligne.

L'ordre des arguments compte : `assertEquals` reçoit l'attendu d'abord, l'obtenu ensuite. En cas d'échec, `JUnit` affiche `expected: <100> but was: <0>`. Si vous les inversez, le message perdra tout son sens.

Enfin, la ligne qui fait le test est la dernière. Sans elle, la méthode appelle du code, ne vérifie rien, et passe au vert : sans assertion, un test ne peut qu'être vert (`CC-TST.4`).

### Les assertions dont vous aurez besoin

| Assertion | Réussit si |
|---|---|
| `assertEquals(attendu, obtenu[, message])` | les deux valeurs sont égales |
| `assertTrue(condition[, message])` | la condition est vraie |
| `assertFalse(condition[, message])` | la condition est fausse |
| `assertNull(valeur[, message])` | la valeur est `null` |
| `assertThrows(Type.class, ... [, message])` | l'appel lève cette exception |

Cinq assertions suffisent à tout ce que le cours et l'examen demandent. Chacune a sa contraire (`assertNotEquals`, `assertNotNull`), et toutes acceptent un argument supplémentaire, un message affiché en cas d'échec.

Pour des objets, `assertEquals` appelle `equals` : un objet qui ne le redéfinit pas n'est égal qu'à lui-même (voir `COL §3`). Pour des `double`, elle demande une tolérance, parce qu'un calcul en virgule flottante n'est presque jamais exact au dernier chiffre : le fascicule de `JUN.3` le montre.

Préférez toujours l'assertion la plus précise. Un `assertTrue(libres == 100)` qui échoue dit seulement « expected: <`true> but` was: <`false`> ». Un `assertEquals(100, libres)` dit ce qu'il a obtenu.

### Un message qui explique l'échec

```
assertEquals(98, salle.placesLibres(),
        "Deux places reservees sur 100 doivent en laisser 98");

// en cas d'echec, JUnit affiche :
// Deux places reservees sur 100 doivent en laisser 98
//     ==> expected: <98> but was: <100>
```

Le dernier argument, entre crochets dans le tableau, est facultatif pour `JUnit`. Dans ce cours, ajoutez-le systématiquement. Il dit ce que le test vérifiait, avec les mots du métier : celui qui lira l'échec, vous-même dans six mois ou un collègue, comprend ce qui s'est cassé sans ouvrir le test. « expected: <98> but was: <100> » dit qu'un nombre est faux ; le message dit lequel, et pourquoi il aurait dû valoir 98.

Le message décrit l'attente, pas le défaut supposé : « doivent en laisser 98 », et non « la réservation est cassée ». Le test ne sait pas pourquoi il échoue ; il sait seulement ce qu'il attendait. Comme pour les noms de variables, pas d'accent dans la chaîne.

### Un test rouge accuse le code, ou le test

```
// une assertion fausse
expected: <98> but was: <100>

// une exception que personne n'attendait
java.lang.UnsupportedOperationException:
    reserver n'est pas encore implementee
```

Un test rouge dit qu'il y a un désaccord, mais ne dit pas qui a tort. Le code peut être faux, mais le test aussi : un attendu mal calculé, des arguments inversés, une préparation incomplète.

L'arbitre n'est ni l'un ni l'autre, c'est la spécification : la documentation de la méthode, ou l'énoncé. On la relit, on décide qui a tort, et on ne corrige que celui-là. Le second cas vous est familier : c'est le message de chacun de vos squelettes. Une exception qui sort d'un test le fait échouer, et sa trace se lit comme au module `EXC`.

C'est le troisième objectif de l'examen : exécuter les tests de quelqu'un d'autre, et s'en servir pour trouver le défaut dans une classe que vous n'avez pas écrite.

> **Exercices** - `JUN.1` Le test comme spécification
>
> C'est le dernier exercice où le test est fourni, et la manière de s'en servir change. Ouvrez `CalculatriceTest` d'abord, et lisez-le comme un énoncé : les signatures, les valeurs, et ce qui doit arriver quand on divise par zéro y sont, et nulle part ailleurs. Vous y croiserez `assertThrows`, que la section suivante explique. La question sur les `assertEquals` multiples mérite d'être confrontée à `CC-TST.2`. Voir aussi `CC-TST.1`.

## JUN §3 — Vérifier tout le contrat

### Une documentation est une liste de tests

```
/**
 * Reserve des places.
 *
 * @throws IllegalArgumentException si places <= 0
 * @throws SalleCompleteException   s'il reste moins de
 *         places que demande ; rien n'est alors reserve
 */
public void reserver(int places)
        throws SalleCompleteException
```

Quand vous testez la classe de quelqu'un d'autre, vous ne partez pas de son code, vous partez de ce qu'elle promet. Chaque phrase de la documentation est une promesse, et chaque promesse mérite son test, y compris celles faites en cas d'échec. Ici, quatre promesses :

1. une réservation diminue les places libres d'autant ;
2. zéro place est refusé ;
3. une demande trop grande est refusée ;
4. ce refus ne réserve rien.

La dernière est la plus souvent oubliée. Elle est écrite en fin de phrase, presque en passant, et c'est pourtant elle qui distingue une salle correcte d'une salle qui réserve quand même la moitié de la demande. C'est ce que l'examen appelle tester « sur base d'une classe implémentée par un tiers et/ou de spécifications techniques » : la spécification, c'est ceci.

### Vérifier qu'une exception est levée

```
@Test
void reserverZeroPlaceEstRefuse() {
    Salle salle = new Salle(100);

    assertThrows(IllegalArgumentException.class,
            () -> salle.reserver(0),
            "Reserver zero place doit etre refuse");
}
```

Appeler `salle.reserver(0)` directement ne marcherait pas : l'exception interromprait le test, qui passerait au rouge précisément quand la salle fait ce qu'on attend d'elle. `assertThrows` fait l'appel pour vous, attrape l'exception, et vérifie son type. Le test échoue si rien n'est levé, ou autre chose.

L'écriture `() -> salle.reserver(0)` emballe l'appel pour qu'`assertThrows` l'exécute elle-même, au bon moment. Pour l'instant, retenez la forme : des parenthèses vides, une flèche, l'appel. Le module `LAM` expliquera cette forme plus en détail. La notation `IllegalArgumentException.class` désigne la classe elle-même.

Nommez le type le plus précis. Une sous-classe convient aussi, si bien qu'`assertThrows(Exception.class, ...)` accepterait n'importe quelle exception, y compris celle d'un bug. `assertThrows` retourne l'exception attrapée : on peut en vérifier le message.

### Le `throws` d'une méthode de test

```
@Test
void deuxPlacesReserveesNeSontPlusLibres()
        throws SalleCompleteException {
    Salle salle = new Salle(100);

    salle.reserver(2);

    assertEquals(98, salle.placesLibres(),
            "Deux places reservees sur 100 doivent en laisser 98");
}
```

Le module `EXC` l'avait annoncé : une méthode de test qui appelle `reserver` doit choisir, attraper ou déclarer (voir `EXC §2`). Dans un test, on déclare. Si l'exception survient alors qu'on ne l'attendait pas, `JUnit` marque le test en rouge avec sa trace : c'est exactement ce qu'on veut.

L'attraper serait pire que tout. Un `catch` vide, ou qui affiche un message, fait passer le test au vert alors que la réservation a échoué. C'est un test sans assertion déguisé, et un `catch` qui ne traite rien, contraire à `CC-ERR.6`. Dans `assertThrows`, en revanche, rien à déclarer : c'est `JUnit` qui attrape.

### Après l'échec, l'état

```
@Test
void unePlaceDeTropNeReserveRien()
        throws SalleCompleteException {
    Salle salle = new Salle(10);
    salle.reserver(8);

    assertThrows(SalleCompleteException.class,
            () -> salle.reserver(3),
            "Une demande trop grande doit etre refusee");

    assertEquals(2, salle.placesLibres(),
            "Le refus ne doit rien reserver");
}
```

Lever la bonne exception ne suffit pas : le refus ne doit rien avoir changé. Une salle qui réserverait les deux places restantes avant de lever l'exception passerait la première assertion ; seule la seconde la démasque.

Les deux assertions ne contredisent pas `CC-TST.2` : elles décrivent un seul comportement, le refus d'une demande trop grande, vu sous ses deux faces. C'est la même idée qu'en `EXC.1`, où le test vérifiait que le point ne bougeait pas quand le déplacement était refusé. Cette fois, c'est vous qui l'écrivez.

> **Exercices** - `JUN.2` Détecter un défaut
>
> L'exercice met la section en pratique sans `JUnit`. Vous n'y écrivez pas de `@Test` : chaque clause du contrat devient une vérification dans une méthode ordinaire, qui retourne `true` ou `false`, avec ce que vous savez depuis le module `EXC`. L'intérêt est que la qualité de votre vérification devient mesurable : chaque tirelire défectueuse n'est démasquée que par une clause précise, et en oublier une, c'est en laisser passer une. Résistez à l'envie d'ouvrir les classes pour trouver le défaut : le repérer en lisant ne prouve rien, et ne servira plus le jour où le code changera.

## JUN §4 — Choisir ses cas, et les justifier

### Où se logent les défauts

On ne peut pas tout tester. On découpe donc les entrées en *familles* qui devraient se comporter de la même façon, on prend un représentant de chacune, et l'on vise les *frontières* entre familles, parce que c'est là que les défauts se logent : un `>` au lieu d'un `>=`, un indice qui commence à 1.

| Famille de cas | Pour une salle de 10 places |
|---|---|
| le cas ordinaire | réserver 3 places |
| la borne atteinte | réserver exactement les places libres |
| la borne dépassée | réserver une place de trop |
| la valeur interdite | réserver 0 place, ou -1 |
| la répétition | deux réservations successives |

Le cas ordinaire est nécessaire et ne suffit jamais : un test qui ne couvre que lui ne protège que de ce qui n'arrive pas. La répétition est la famille la plus oubliée : beaucoup de défauts n'apparaissent qu'au deuxième appel, quand l'objet n'est plus neuf.

Le fascicule de `JUN.3` donne la liste des cas de `Moyenne`. Ce tableau est la méthode qui produit une telle liste quand personne ne vous la donne.

### Un cas, un test, un nom

```
@Test
@DisplayName("Reserver toutes les places libres remplit la salle")
void reserverToutesLesPlacesLibres()
        throws SalleCompleteException {
    Salle salle = new Salle(10);

    salle.reserver(10);

    assertEquals(0, salle.placesLibres(),
            "Reserver les 10 places doit remplir la salle");
}
```

Une méthode par comportement (`CC-TST.2`), et un nom qui dit ce qui est vérifié, jamais un numéro (`CC-NOM.1`). Quand ce test passera au rouge, son nom sera la première chose que vous lirez, et souvent la seule dont vous aurez besoin. `test1` ne dit rien. L'ancienne habitude de commencer chaque nom par `test` n'apporte rien non plus : l'annotation le dit déjà.

L'annotation `@DisplayName` énonce le test en clair, avec des espaces ; IntelliJ l'affiche à la place du nom de la méthode. Pas d'accent dans la chaîne, comme partout dans le code du cours. `JUN.3` exige un `@DisplayName` sur chaque test, et refuse les noms numérotés.

Une exception : si l'énoncé impose un nom, on le respecte à la lettre. L'examen de 2022 imposait `testConcatene`, et la correspondance exacte à l'énoncé fait partie des critères.

### Une valeur, un test

```
@Test
void reserverUnePlaceEnLaisseNeuf()
        throws SalleCompleteException {
    Salle salle = new Salle(10);

    salle.reserver(1);

    assertEquals(9, salle.placesLibres(),
            "Une place reservee sur 10 doit en laisser 9");
}

// un autre test pour 5 places, un autre pour 10
```

La question revient chaque année : faut-il vérifier plusieurs valeurs dans une même méthode de test, ou écrire une méthode par valeur ? La règle est une méthode par valeur. Une assertion qui échoue arrête la méthode : si la première de cinq valeurs est fausse, les quatre autres ne sont jamais vérifiées, et vous corrigez à l'aveugle. Et le nom de la méthode ne dit pas quelle valeur a cassé, alors qu'un test par valeur le dit d'un coup d'œil.

La règle connaît des exceptions raisonnables. Après une seule action, on peut vérifier plusieurs choses sur son résultat : « Après l'échec, l'état » vérifie l'exception, puis que rien n'a changé. Et un test qui contrôle la forme d'une classe peut vérifier tous ses champs d'un coup. Ce qu'on évite, c'est d'enchaîner plusieurs actions indépendantes dans une seule méthode.

> **Pour aller plus loin** - `@ParameterizedTest`
>
> Quand une même vérification doit porter sur beaucoup de valeurs, `JUnit` propose un compromis : une seule méthode, lancée une fois par cas, et rapportée comme autant de tests séparés. Avec `@ParameterizedTest` et `@CsvSource({"1, 9", "5, 5", "10, 0"})`, la méthode reçoit en paramètres l'entrée et l'attendu de chaque ligne. Si le cas « 5, 5 » échoue, les deux autres sont quand même vérifiés. Les annotations viennent de `org.junit.jupiter.params`, déjà présent dans le projet. Rien n'oblige à s'en servir : une méthode par valeur reste toujours correcte.

### Chaque test repart de zéro

```
class SalleTest {

    private Salle salle;

    @BeforeEach
    void preparer() {
        salle = new Salle(10);
    }

    @Test
    void uneSalleNeuveEstEntierementLibre() {
        assertEquals(10, salle.placesLibres(),
                "Une salle neuve doit etre libre");
    }
}
```

`JUnit` crée un nouvel objet de test pour chaque méthode, et la méthode annotée `@BeforeEach` s'exécute juste avant elle. Un test qui remplit la salle ne gêne donc pas le suivant. `JUnit` ne promet aucun ordre d'exécution : un test qui a besoin qu'un autre ait tourné avant lui finira par échouer sans raison apparente. Aucun test ne compte sur un autre (`CC-TST.3`).

N'en abusez pas. Si la valeur qui compte pour un test est cachée dans `preparer`, le lecteur doit remonter la chercher. Ce qui fait le cas d'un test reste visible dans le test.

### Un test se justifie par le défaut qu'il attrape

```
// le defaut :  >=  au lieu de  >
if (places >= placesLibres()) {
    throw new SalleCompleteException(
            "Plus que " + placesLibres() + " places");
}

// un seul test passe au rouge :
// reserverToutesLesPlacesLibres
```

Justifier un test, c'est nommer le défaut qu'il attraperait. Ici, la salle refuse une demande qu'elle pouvait servir. Le cas ordinaire ne le voit pas, la place de trop non plus : seul le test de la borne atteinte passe au rouge. C'est sa justification, en une phrase.

La question se pose dans les deux sens. Un test pour lequel on ne trouve aucun défaut qu'il attraperait est décoratif. Deux tests qui attrapent exactement les mêmes défauts en font un de trop. C'est le quatrième objectif de l'examen, « justifier ses choix quant aux tests implémentés » : la réponse la plus solide ne dit pas « pour tester le cas limite », elle dit quelle erreur ce cas limite démasque.

### À l'examen : écrire, lancer, corriger

La troisième question de l'examen a toujours la même forme : une classe fournie, une classe de test à écrire, puis lancer les tests. La classe fournie peut contenir une erreur, à corriger si vos tests la trouvent.

- **Le plus souvent, l'énoncé dicte les cas :** vous traduisez chaque phrase en méthode, comme en `JUN §3`. Quand il ne dit que « envisagez les différents cas possibles », c'est le tableau des familles de cas qui répond.
- **Vos tests peuvent révéler un défaut : c'est prévu.** Revenez alors à « Un test rouge accuse le code, ou le test ».
- **Si l'énoncé vous donne raison, la classe est fausse :** corrigez-la, et relancez tout. Modifier l'attendu pour retrouver le vert, c'est rendre un test qui ment.

> **Exercices** - `JUN.3` Écrire une classe de test
>
> L'exercice est la forme exacte de la question d'examen, sans la correction : la classe `Moyenne` est juste. Vous écrivez `MoyenneTest`, au moins cinq tests, chacun avec son `@DisplayName`. `RedactionTest` vérifie que votre classe existe et qu'elle a la bonne forme ; elle ne peut pas juger si vos tests sont pertinents. Ce jugement-là, c'est vous qui le portez. Faites l'expérience proposée par le fascicule : glisser un défaut dans `Moyenne`, et regarder si vos tests le voient. C'est la seule mesure honnête de ce que vous avez écrit. Voir `CC-TST.1`, `CC-TST.2` et `CC-TST.3`.

## JUN §5 — Du code qui se laisse tester

### Pourquoi certaines classes résistent aux tests

Un test prépare une situation, déclenche une action, vérifie le résultat : seul, et en un instant. Certaines classes l'en empêchent.

- **Une classe qui ouvre une fenêtre** ne peut pas tourner dans un test : il faudrait que quelqu'un regarde l'écran.
- **Une classe qui lit la date du jour** donne un résultat qui change d'une année à l'autre : un test juste aujourd'hui sera faux demain.
- **Une classe qui tire au hasard** ne donne jamais deux fois la même chose : que vérifier ?

Dans les trois cas, ce n'est pas le test qu'il faut changer, c'est la classe. Le remède est dans la classe elle-même : une interface à la place de la fenêtre, un paramètre à la place de la date ou du hasard. Retenez l'idée générale : une classe difficile à tester est souvent une classe mal découpée.

### Une doublure à la place de la fenêtre

Au module `ITF`, le contrôleur ne dépend plus que d'une interface, `Afficheur`, qu'il reçoit par `setAfficheur` (voir `ITF §4`). Qui affiche, et comment, le contrôleur l'ignore. C'est exactement cette ignorance qui rend le test possible.

```
class AfficheurQuiRetient implements Afficheur {

    String dernier;

    @Override
    public void affiche(String message) {
        dernier = message;
    }
}
```

Pour le test, on écrit un faux afficheur : il implémente `Afficheur`, mais au lieu d'afficher, il range le message dans un champ, que le test pourra lire. On appelle ce genre de remplaçant une *doublure*, comme au cinéma : elle tient le rôle à la place de la vraie, le temps d'une scène. Elle ne sert qu'au test, et vit donc dans `src/test`.

```
@Test
void leControleurSignaleLEvenement() {
    Controleur controleur = new Controleur();
    AfficheurQuiRetient afficheur = new AfficheurQuiRetient();
    controleur.setAfficheur(afficheur);

    controleur.annoncer("bouton presse");

    assertEquals("Evenement : bouton presse", afficheur.dernier,
            "Le controleur doit transmettre l'evenement a l'afficheur");
}
```

Le test suit les trois temps habituels : préparer un contrôleur et lui brancher la doublure, annoncer un événement, vérifier que la doublure a reçu le bon message. Aucune fenêtre ne s'ouvre, et le test tourne en quelques millisecondes. Le contrôleur n'a pas changé d'une ligne pour être testé : c'est le même que celui qui pilotera la vraie fenêtre. Sans l'interface du module `ITF`, ce test serait impossible.

### Ni le jour, ni le hasard

Au module `POO`, l'âge d'un livre n'était pas stocké : `getAge` le calculait à partir de la date du jour. Juste, mais difficile à tester : pour *Dune*, paru en 1965, faut-il attendre 61 ou 62 ? Cela dépend du jour où le test tourne. Un test écrit aujourd'hui deviendra rouge le premier janvier, alors que personne n'aura touché au code.

```
public int getAge(int anneeCourante) {           // testable
    return anneeCourante - anneeDeParution;
}
public int getAge() {                            // pratique
    return getAge(LocalDate.now().getYear());
}
```

```
assertEquals(61, dune.getAge(2026),     // vrai aujourd'hui, et dans dix ans
        "Dune, paru en 1965, a 61 ans en 2026");
```

On sépare le calcul de la commodité. Le calcul reçoit l'année en paramètre : le test la fixe, et le résultat ne bouge plus. La version sans paramètre, celle qu'on appelle d'habitude, se contente d'aller chercher l'année et de déléguer. Vous avez déjà fait ce découpage au module `POO`, avec `computeAge(int annee)`.

Le hasard se traite de la même façon : la classe reçoit son `Random` au lieu de le créer, et le test lui en donne un avec une graine, qui tire toujours les mêmes nombres (voir la note sur le générateur pseudo-aléatoire en `ENU §6`). Le mémento Clean Code montre l'exemple complet en `CC-TST.3`.

## Ce qu'il faut retenir

- Un test contient la réponse attendue, et vérifie seul ; sans assertion, il ne vérifie rien.
- Trois temps : 1. préparer, 2. agir, 3. vérifier ; `assertEquals(attendu, obtenu, message)`, dans cet ordre.
- Une documentation est une liste de tests, y compris pour les échecs : `assertThrows`, puis l'état.
- Dans un test, on déclare les exceptions ; on ne les attrape jamais.
- Des familles de cas, leurs frontières, et un test par valeur, au nom qui dit ce qu'il vérifie.
- Un test se justifie par le défaut qu'il attrape.
- Une classe difficile à tester est souvent mal découpée : une interface, ou un paramètre, la rend testable.

La leçon qui dépasse ce module : un test rouge ne dit pas qui a tort. L'arbitre est la spécification, et c'est elle qu'on relit avant de corriger quoi que ce soit.